



BITS Pilani

Pilani | Dubai | Goa | Hyderabad | Mumbai

**WORK INTEGRATED
LEARNING PROGRAMMES**

AI-Powered Merge Request Risk and Code Review Decision Support System

MAIP Experiential Learning Assignment

Name: M Mahadev

Student ID: 2025mb26043

Programme: MBAZG582

Course: MAIP

Academic Year: 2026

Phase 1 – Problem Identification, Diagnosis and Analytical Framing

1. Organizational / Business Context

Modern software engineering organizations operate with large numbers of Merge Requests (MRs) across multiple repositories, services and development teams. Every MR potentially requires code review, testing, security assessment and release validation before it can be merged into the main codebase.

However, all Merge Requests do not have the same level of risk. A documentation change or minor configuration update may require limited review, whereas a change involving authentication, payment processing, database access, infrastructure, networking, secrets or security-sensitive components may have significant consequences if defects are introduced.

In many development environments, review priority is largely determined by the size of the change, the developer's experience, manual reviewer judgment or simple repository rules. This can make it difficult for engineering teams to consistently identify high-risk changes early.

The proposed project addresses this problem by investigating an **AI/ML-based Merge Request Risk and Code Review Decision Support System**. The system would analyze the characteristics and content of Merge Requests and provide a risk assessment, explain the major risk factors and recommend the appropriate level of review or testing.

The project therefore focuses not merely on automating code review, but on helping engineering managers, technical leads and reviewers **allocate limited review and testing resources to the changes that require the greatest attention**.

2. Problem Statement

Engineering teams need to review a large number of Merge Requests while operating under limited developer, senior-engineer and security-review capacity.

The current review process may not consistently distinguish between low-risk and high-risk software changes. As a result, reviewers may spend significant time on relatively low-risk changes while potentially high-risk changes may not receive sufficient attention.

A high-risk Merge Request may contain changes to authentication, authorization, infrastructure, database logic, dependencies or security-sensitive configuration. It may also introduce vulnerabilities such as exposed credentials, insecure configurations, unsafe code patterns or insufficient testing.

The core problem is therefore:

Engineering teams lack a systematic, data-driven method for predicting the risk of Merge Requests and prioritizing the appropriate level of code review, security assessment and testing effort.

The proposed solution is an AI-assisted decision-support system that analyzes Merge Requests and predicts their potential risk while providing interpretable explanations and recommendations.

3. Decision to be Made

The primary managerial decision is:

Which Merge Requests require additional engineering, security or testing attention before they can be safely merged?

Based on the predicted risk, the system could support decisions such as:

- Whether an MR should receive normal or enhanced review.
- Whether a security specialist should review the MR.
- Whether additional testing should be required.
- Whether a senior engineer should review the change.
- Which MRs should receive priority when reviewer capacity is limited.
- Whether an MR contains characteristics associated with previous defective or risky changes.
- Whether an AI-generated recommendation should trigger additional human investigation.

The final objective is **not to automatically approve or reject code**. Instead, the system would act as a decision-support mechanism for human reviewers and engineering management.

This is aligned with the assignment's requirement that the problem should involve a real decision that an organization needs to make.

4. Problem Diagnosis

Major Symptoms

The problem may manifest through several operational symptoms:

1. **Unequal review effort:** Large amounts of reviewer time may be spent on changes that have relatively low risk.
2. **Late identification of security issues:** Vulnerabilities or exposed credentials may only be discovered during later security scans, testing or production incidents.
3. **Inconsistent review quality:** The depth of review can depend heavily on individual reviewer experience and judgment.
4. **Insufficient testing for high-impact changes:** Changes to critical components may not always receive proportionate testing.
5. **Difficulty prioritizing MRs:** When many MRs are waiting for review, engineering teams may lack a systematic mechanism for determining which changes deserve immediate attention.

Possible Underlying Causes

Potential causes include:

- Large variation in MR size and complexity.
- Different levels of developer and reviewer experience.
- Lack of standardized risk classification.
- Increasing software and architectural complexity.
- Changes spanning multiple services or components.
- Security-sensitive code being mixed with routine changes.
- Limited availability of senior engineers and security specialists.
- Reliance on manual inspection.
- Limited use of historical MR information when evaluating new changes.

Factors That May Influence Risk

Potential risk factors include:

- Number of files modified.
- Lines of code added or removed.
- Type of files modified.
- Programming language.
- Code complexity.
- Number of dependencies changed.
- Configuration changes.
- Authentication or authorization changes.
- Database modifications.
- Infrastructure changes.
- Security-sensitive components.
- Number of tests added or modified.
- Historical defects associated with similar changes.
- Previous review comments or rework.
- Repository/component criticality.

At Phase 1, these are proposed factors rather than validated findings. The subsequent analysis will determine which factors actually demonstrate meaningful predictive value.

5. Managerial Importance

The problem has direct implications for **engineering productivity, software quality, operational risk and security**.

Cost and Productivity

Senior developers and security specialists are limited resources. If their time can be allocated more effectively toward high-risk changes, organizations may improve review efficiency without simply increasing headcount.

Operational Risk

Defects introduced through software changes can result in service degradation, failed deployments, production incidents or customer-impacting issues.

Security Risk

Software changes can introduce vulnerabilities or accidentally expose sensitive information such as passwords, API keys, tokens or private credentials. Early identification can reduce the likelihood of security incidents.

Quality

A systematic risk assessment can encourage proportionate testing and review. High-risk changes can receive additional validation while low-risk changes can move through the development process more efficiently.

Management Visibility

An aggregated risk view could help engineering managers understand:

- How much high-risk development activity is occurring.
- Which services generate the highest-risk changes.
- Where review capacity is required.
- Which types of changes are associated with defects.
- Whether additional security or testing resources may be necessary.

Therefore, the proposed solution has relevance not only for individual developers but also for **engineering management and organizational resource allocation**.

The assignment specifically asks that managerial importance be considered in terms such as cost, operations, risk, reputation and strategy.

6. Proposed AI-Based Solution

The proposed system will act as an **AI-powered Merge Request analysis and decision-support platform**.

At a high level:

Merge Request → Feature Extraction → ML Risk Prediction → Code/Security Analysis → AI Explanation → Review Recommendation

The system could analyze both structured and unstructured information contained in an MR.

A. ML-Based Risk Prediction

A Machine Learning model could classify an MR into categories such as:

- Low Risk
- Medium Risk
- High Risk

Alternatively, the system could estimate a continuous risk score.

Possible input characteristics include MR size, code complexity, files changed, component type, testing information, dependency changes and historical characteristics.

Candidate techniques could include:

- Logistic Regression
- Decision Trees
- Random Forest
- Gradient Boosting
- Other appropriate classification models

The final model should be selected based on the characteristics of the faculty-provided data rather than assuming a particular algorithm in advance.

B. AI-Based Code Analysis

A Generative AI/LLM component could analyze the actual code changes and provide a human-readable explanation of the MR.

For example:

"The MR modifies JWT authentication logic and changes security-sensitive configuration. No corresponding authentication tests were identified. Additional security and integration testing is recommended."

C. Security Analysis

The system could identify or prioritize potential issues such as:

- Hard-coded passwords.
- API keys and tokens.
- Private credentials.
- Unsafe configuration.
- SQL injection patterns.
- Command injection risks.
- Insecure authentication logic.
- Excessive container privileges.
- Sensitive information committed to repositories.

Traditional security tools could potentially complement the AI component rather than replacing established security scanning mechanisms.

D. Test Risk Analysis

The system could compare the nature of the changes with testing activity.

For example:

"Database transaction logic was modified, but no corresponding tests were added or modified. Testing risk is therefore elevated."

E. Review Recommendation

Based on the predicted risk and analysis, the system could recommend:

- Standard code review.
- Senior developer review.
- Security review.
- Infrastructure/platform review.
- Additional automated testing.
- Additional integration testing.

7. Potential User Interface / Deployment

The proposed system could eventually be integrated into existing development workflows.

A Git-based integration could analyze an MR when it is created or updated and provide a summary directly within the MR.

A future IDE/VS Code extension could allow developers to analyze changes before creating the MR.

The IDE integration is therefore considered a **delivery mechanism for the AI decision-support capability**, rather than the primary problem itself.

A conceptual workflow is:

Developer → Git Repository / IDE → MR Analysis Engine → ML Model + Security Analysis + GenAI → Risk Assessment → Reviewer / Engineering Manager

8. Stakeholders

The key stakeholders are:

Developers

Receive early feedback about security, quality and testing concerns before submitting or merging changes.

Code Reviewers / Technical Leads

Receive a prioritized assessment of which areas of the MR require deeper investigation.

Security Teams

Can identify MRs that modify security-sensitive functionality and may require additional security review.

Engineering Managers

Can prioritize limited review capacity and understand risk patterns across repositories and teams.

Release / DevOps Teams

Can use risk information to identify changes that may require additional validation before deployment.

Organization

May benefit from reduced defect risk, improved engineering efficiency and better allocation of technical resources.

The assignment requires stakeholder identification as part of the problem framing.

9. Constraints and Trade-offs

Several constraints need to be considered.

Data Availability

The effectiveness of the ML model depends on the availability of historical MR information and meaningful outcome variables such as defects, review outcomes or production issues.

Data Privacy

Source code and Merge Requests may contain confidential business logic, customer information or credentials. Any AI solution must therefore consider data privacy and secure processing.

False Positives

An overly sensitive system could flag many legitimate changes as risky, resulting in alert fatigue and reducing trust in the system.

False Negatives

Failure to identify an actually risky MR could create a false sense of security. Therefore, AI recommendations should not replace human judgment.

Explainability

Engineering teams need to understand why an MR was classified as high risk. Explainable AI techniques and clear risk factors will therefore be important.

Model Bias

Historical review and defect data may reflect existing organizational practices and biases. A model trained directly on such data could reproduce those patterns.

Implementation Capability

Integration with existing Git platforms, CI/CD systems and IDEs may require additional engineering effort.

These considerations are important because the assignment specifically requires analysis of constraints such as data, technology, privacy, people, fairness and implementation capability.

10. Analytical Framing

The proposed project can involve multiple analytical approaches.

Primary Analytical Problem: Classification

Predict whether an MR belongs to a:

Low / Medium / High Risk

category.

Secondary Analytical Problem: Prediction

Predict the likelihood that a change may require additional review effort or may be associated with a defect, depending on the available faculty-provided data.

Text Analysis

Use textual information from:

- MR titles.
- MR descriptions.
- Commit messages.
- Review comments.

to identify relevant patterns.

Generative AI

Use an LLM to:

- Summarize changes.
- Explain predicted risks.
- Identify potential security concerns.
- Generate review recommendations.
- Recommend areas requiring additional testing.

Explainable AI

Identify the features that contribute most strongly to the predicted risk.

For example:

High Risk because authentication files were modified, code complexity increased and no corresponding tests were added.

The assignment explicitly identifies classification, text analysis, anomaly detection, Generative AI and AI-assisted decision support as relevant analytical approaches.

11. Key Analytical Questions

The subsequent analysis should attempt to answer the following questions:

1. **Can historical Merge Request characteristics be used to predict whether a new MR is low, medium or high risk?**
 2. **Which characteristics of a Merge Request contribute most significantly to its predicted risk?**
 3. **Can AI-assisted code analysis identify security, quality and testing concerns that require additional human review?**
 4. **Can the proposed system help engineering teams prioritize reviewer and security resources more effectively?**
 5. **Can AI-generated explanations and recommendations make the predicted risk sufficiently understandable and actionable for engineering decision-makers?**
-

12. Expected Managerial Outcome

The expected outcome is not simply a model that produces a prediction.

The ultimate objective is a **decision-support framework for software engineering organizations**.

For example:

Predicted Risk	Suggested Management Action
Low	Standard review process
Medium	Additional reviewer attention
High	Senior/security review + additional testing

The system would allow management to move from:

"We have 100 MRs waiting for review."

to:

"These 15 MRs have characteristics associated with elevated risk and should receive priority review."

This can potentially improve the allocation of scarce engineering and security resources while reducing software quality and security risks.

13. Scope for Future Development

If the initial analysis demonstrates sufficient evidence, the solution could later be extended with:

- GitLab/GitHub integration.
- VS Code and IDE plugins.
- Automated MR comments.
- Reviewer recommendation.
- Review-effort prediction.
- Historical similar-MR analysis.
- Change-impact analysis.
- Automated test-case recommendations.
- Engineering risk dashboards.
- Continuous model monitoring and MLOps.

These extensions would be considered after the core analytical problem has been validated.

14. Conclusion

Merge Request review is an important software engineering process that requires organizations to balance development velocity, software quality, security and limited engineering resources.

The proposed **AI-Powered Merge Request Risk and Code Review Decision Support System** addresses this challenge by combining Machine Learning, Generative AI, code analysis and explainable recommendations.

The central objective is to predict the relative risk of software changes and help engineering teams determine **which changes require additional review, security assessment or testing effort**.

Rather than replacing human reviewers, the proposed solution is intended to augment their decision-making by providing consistent, data-driven risk signals and actionable explanations.

The project therefore represents an **AI-assisted managerial decision-support problem**, while also providing a practical pathway toward integration with existing Git-based development workflows.

Importantly, the next stage should be based on the dataset, scenario or evidence package provided by the faculty. Phase 1 should remain focused on defining and framing the problem rather than prematurely collecting data, creating synthetic data or training models, as explicitly required by the project guidelines.